


```
docker compose up -d
docker compose logs proxy      # logs of one service by its name
```

- "8080:80" — external 8080 is what curl/browser use, internal 80 is what nginx listens on. That pair is the only reason `localhost:8080` works.
- `docker compose logs proxy` uses the *service name*, not the container name.
- Indentation is spaces, never tabs.

Step 2 — Explore: file system, permissions, processes, logs (15 min)

```
docker exec -it nginx-explore bash
find / -name "nginx.conf" 2>/dev/null
cat /etc/nginx/conf.d/default.conf
touch /tmp/testfile.txt
chmod 600 /tmp/testfile.txt
ls -la /tmp/testfile.txt      # permissions: rw-----
exit

docker exec nginx-explore ps aux | grep nginx      # processes
docker exec nginx-explore cat /var/log/nginx/access.log  # logs
```

- `chmod 600` makes a file readable/writable by its owner only — read `ls -la` to see the change.
- `ps aux` shows the few processes a container runs — not a whole OS.

Step 3 — Reach it two ways (10 min)

```
curl http://localhost:8080      # raw HTML of the welcome page
# browser on the host: http://localhost:8080 -> "Welcome to nginx!"
```

Run `curl` once more, then read the access log again — your request now appears as a `GET` line.

Step 4 — Stop and remove (5 min)

```
docker stop nginx-explore && docker rm nginx-explore
```

Question: which part of `-p 8080:80` makes `localhost:8080` reach nginx's port 80?

Daily Goals

- Make one class inherit from another with `: BaseClass`
- Call the base constructor from a derived constructor with `base(...)`
- Explain the `is-a` relationship and when inheritance is the wrong tool
- Use `virtual` / `override` to let a derived class replace base behavior
- Write an `abstract` class with an abstract method every derived kind must implement
- Hold mixed kinds in one base-typed `List<>` and call one method polymorphically
- Explain abstraction and polymorphism and point to where you built them

Schedule

Time	Block	Activity
09:00 – 09:40	Morning	Docker block (above)
09:40 – 10:15	Opening	Instructor — many kinds of one thing; the base class
10:30 – 11:30	Keywords, self-investigation, examples, comprehension	Read links, fill keyword table, answer the "why" questions, read and run the code examples, answer the comprehension questions
11:30 – 12:45	Long exercise — planning	Start the daily exercise; this block is mostly planning your class hierarchy
14:45 – 17:45	Long exercise — build	Continue building the daily coding exercise

Keywords (10:30 – 11:30)

Term	Definition	Example
inheritance	A class taking the fields and methods of another	<code>class Aircraft : Track</code>
base class	The shared parent class	<code>Track</code> is the base of <code>Aircraft</code>
derived class	The child that inherits	<code>Aircraft</code> derives from <code>Track</code>
<code>is-a</code> relationship	Inheritance is right only when child <i>is a</i> kind of parent	an aircraft <i>is a</i> track

Term	Definition	Example
<code>base(...)</code>	Calls the base class constructor from a derived one	<code>: base(id, speed)</code>
<code>protected</code>	Visible to derived classes but not outside code	<code>protected double _x;</code>
<code>virtual</code>	Marks a base method as overridable	<code>public virtual string Describe()</code>
<code>override</code>	Replaces a <code>virtual</code> / <code>abstract</code> base method	<code>public override string Describe()</code>
<code>abstract class</code>	A base that cannot be instantiated	<code>abstract class Track</code>
<code>abstract method</code>	A method with no body the derived class <i>must</i> implement	<code>public abstract string Describe();</code>
<code>sealed</code>	Stops further inheritance	<code>sealed class Drone : Aircraft</code>
abstraction	Modeling the shared idea without implementing the specifics	the abstract <code>Track</code>
polymorphism	One call, different behavior depending on the object's real type	<code>List<Track></code> each describing itself

Self-study links:

-
- C# polymorphism (Microsoft docs): <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism>
- C# inheritance (Microsoft docs): <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/inheritance>
- C# abstract classes (Microsoft docs): <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/abstract>
- `virtual` / `override` (Microsoft docs): <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/virtual>
- C# inheritance (W3Schools): https://www.w3schools.com/cs/cs_inheritance.php

Self-Investigation (10:30 – 11:30)

Answer in your own words — no code needed yet.

1. Three kinds of track all have id, speed, and heading; only some have an altitude. If you wrote three separate classes, what would you have to repeat in each? What is the cost of that repetition when a shared field changes?
 2. Inheritance is right when the child *is a* kind of the parent. Give one example where inheritance looks tempting but the `is-a` test fails — where it is really "has-a" or just shared code.
 3. Why do you think the language forbids creating an instance of an `abstract` class? What would go wrong if it let you?
 4. Last week, different behavior for different statuses meant an `if / switch` on a string. With `override`, each object "knows" its own behavior. What does the type system do for you that the switch did by hand?
 5. You will put `Aircraft` and `Vessel` objects into one `List<Track>`. Why is that allowed, and how can one identical line of code print different output for each?
-

Code Examples (10:30 – 11:30)

Example 1 — Inheritance and `base(...)`

```
class Track
{
    public int Id { get; }
    public double Speed { get; set; }
    public Track(int id, double speed)
    {
        Id = id;
        Speed = speed;
    }
}

class Aircraft : Track           // Aircraft IS-A Track
{
    public double Altitude { get; }
    public Aircraft(int id, double speed, double altitude)
        : base(id, speed)        // build the Track part first
    {
        Altitude = altitude;    // then the part unique to Aircraft
    }
}

Aircraft a = new Aircraft(1, 420, 30000);
```

```
Console.WriteLine($"{a.Id} {a.Speed} {a.Altitude}"); // Id and Speed
inherited
```

Line by line:

- `class Aircraft : Track` means `Aircraft` inherits `Id` and `Speed` — you do not re-declare them.
- `: base(id, speed)` forwards the shared fields to the `Track` constructor. Without it, the base part is not built.
- `a.Id` works because it was inherited.

How to run: create an `Aircraft`, print all three values, confirm `Id / Speed` came from the base.

Example 2 — virtual and override

```
class Track
{
    public int Id { get; }
    public Track(int id) { Id = id; }

    public virtual string Describe() // virtual: a derived class MAY
replace it
    => $"Track {Id}";
}

class Aircraft : Track
{
    public double Altitude { get; }
    public Aircraft(int id, double altitude) : base(id) { Altitude = altitude;
}

    public override string Describe() // override: this kind replaces the
behavior
    => $"Aircraft {Id} at {Altitude} ft";
}
```

Line by line:

- `virtual` on the base method gives permission to replace it.
- `override` in the derived class provides the replacement.
- Without `virtual` (or `abstract`) on the base, `override` will not compile.

How to run: call `Describe()` on a `Track` and on an `Aircraft`, compare the output.

Example 3 — An abstract class that forces each kind to describe itself

```
abstract class Track // abstract: a Track "in general"
cannot exist
{
    public int Id { get; }
    public double Speed { get; set; }
    protected Track(int id, double speed) // base constructor for derived
classes
    {
        Id = id;
        Speed = speed;
    }
    public abstract string Describe(); // no body – every kind MUST
implement it
}

class Aircraft : Track
{
    public double Altitude { get; }
    public Aircraft(int id, double speed, double altitude)
        : base(id, speed) => Altitude = altitude;
    public override string Describe()
        => $"Aircraft {Id} at {Altitude} ft, {Speed} kn";
}

class Vessel : Track
{
    public Vessel(int id, double speed) : base(id, speed) { }
    public override string Describe()
        => $"Vessel {Id}, {Speed} kn";
}

// Track t = new Track(1, 100); // <- compile error: cannot create an
abstract class
//Severity Code Description Project File Line Suppression State
//Error (active) CS0144 Cannot create an instance of the abstract type or
//interface 'Track' W2D2 D:\C#Projects\W2D2\Program.cs 34

Aircraft a = new Aircraft(1, 420, 30000); // concrete kinds are fine
```

Line by line:

- `abstract class Track` cannot be `new`-ed — try it and read the error.
- `public abstract string Describe();` has no body. Every derived class is forced to provide one.
- `Track` models the *idea* of a track without saying how any specific kind describes itself — that is **abstraction**.

How to run: try `new Track(...)` and read the compile error, then create concrete `Aircraft` and `Vessel` objects.

Example 4 — Polymorphism: one list, many kinds, one call

```
Aircraft a = new Aircraft(1, 300, 30000);    // concrete kinds are fine
Vessel b = new Vessel(1, 300);              // concrete kinds are fine

Track[] allArr = new Track[3]; // arr of tracks holds different kinds of tracks
allArr[0] = a;
allArr[1] = b;
allArr[2] = new Aircraft(1, 2, 3);

for (int i = 0; i < allArr.Length; i++) {
    Console.WriteLine(allArr[i].Describe()); }

List<Track> all = new()
{
    new Aircraft(1, 420, 30000),
    new Vessel(2, 18),
    new Aircraft(3, 510, 41000)
};

foreach (Track t in all)
    Console.WriteLine(t.Describe()); // each object responds in its own way
```

Line by line:

- One `List<Track>` holds different concrete kinds — allowed because each *is-a* `Track`.
- The loop is identical for every object, yet each prints its own description.
- The object's **real type** decides which `Describe()` runs — not the type of the variable. That is **polymorphism**.

- Compare with last week: different behavior used to mean a `switch` on a status string. Now the type system picks the behavior.

How to run: run the loop, confirm aircraft and vessel lines look different, then add a third kind and watch it just work.

Comprehension Questions (10:30 – 11:30)

1. (multiple choice) What does `class Aircraft : Track` mean?
a) Aircraft contains a Track b) Aircraft inherits from Track c) Track inherits from Aircraft d) they are unrelated
 2. (multiple choice) What does `base(id, speed)` do in a derived constructor?
a) creates a new base object b) calls the base class constructor c) overrides a method d) nothing
 3. (multiple choice) What happens if you try `new Track(...)` when `Track` is `abstract` ?
a) it works b) it returns null c) it fails to compile d) it creates an Aircraft
 4. (open) Explain polymorphism using the `List<Track>` loop. Why does one identical line print different output per object?
 5. (open) When is inheritance the right tool, and when is it just shared code that fails the `is-a` test? Give one example of each.
-

Coding Exercise (11:30 – 12:45 planning, 14:45 – 17:45 build)

Intelligence need: A tracking cell follows many platforms at once — fast-movers in the air, slow contacts at sea, vehicles on the ground. They all report position and speed the same way, but each platform type carries one extra detail that only makes sense for it, produces a different status line for the watch officer's screen, and decides by its own rule whether the track on it is still reliable. The cell needs one list it can walk down once — asking every platform to report itself and to say whether it is still trackable — without the code caring which type each one is.

Before writing any code: plan your class hierarchy on paper. Decide what every platform shares (goes in the base), what each type adds that is unique to it (one field per type), and the two methods every type must implement differently. Only open the editor once the hierarchy is drawn.

Title: A Tracked Platform Hierarchy

Build a console program with one abstract base type and exactly three derived platform types, exercised polymorphically through a single base-typed list.

Functional requirements:

- Define an **abstract** base class `Platform` with a `protected` constructor. It holds only what every platform shares: an `int TrackId`, a `double SpeedKnots`, and a `double Heading`.
- `Heading` must be exposed through a **validating property**: a value outside 0–359 is rejected or corrected at the gate (reuse yesterday's encapsulation). `SpeedKnots` must never be negative. `TrackId` is read-only after construction. Backing data is `private` / `protected`; **no public fields**.
- Give the base **exactly two abstract methods** that every derived type must implement, and nothing else abstract:
 - `string StatusLine()` — a one-line description for the watch officer's screen.
 - `bool IsTrackable()` — whether the track on this platform is still reliable, decided by each type's own rule using its unique field.
- Write **exactly three** derived classes, each calling `base(...)` from its constructor:
 - `AirPlatform` — adds **one** unique field `double AltitudeFeet`. `IsTrackable()` is `true` only when `AltitudeFeet` is between 100 and 60000 **and** `SpeedKnots > 0`.
 - `SeaPlatform` — adds **one** unique field `double DepthMeters` (0 = surface). `IsTrackable()` is `true` only when `DepthMeters` is between 0 and 300 (deeper than that, the contact is lost).
 - `GroundPlatform` — adds **one** unique field `string TerrainType`. `IsTrackable()` is `true` only when `TerrainType` is not "tunnel" (a platform that entered a tunnel cannot be tracked).
- Each derived class provides its **own override** of both `StatusLine()` and `IsTrackable()`. `StatusLine()` must include the type's unique field, so the three status lines are visibly different from one another.
- Add a `ToString()` override (on the base, or per type) that prints the shared status: `TrackId`, `SpeedKnots`, `Heading`.
- Create a single `List<Platform>` holding **at least six** objects, with **all three** types represented and **at least one** object of each type that is *not* trackable. Loop over the list **once** and, for each platform, print its `StatusLine()` together with whether it `IsTrackable()`. Each platform must report and decide in its own way. **No `if` / `switch` on a type or a status string to choose behavior** — the type system decides which `StatusLine()` and which `IsTrackable()` run.

Use this domain as given. Do not copy the code examples from earlier today; they describe their objects differently and use different field names — build your own from the requirements above.

Common Errors

Error	Cause	Fix
There is no argument given that corresponds to the required parameter	Forgot <code>base(...)</code> in the derived constructor	Forward shared fields with <code>: base(...)</code>
cannot declare instance of abstract class	Tried to <code>new</code> the abstract base	Create a concrete derived kind instead
no suitable method found to override	Base method is not <code>virtual</code> or <code>abstract</code>	Mark the base member <code>virtual</code> or <code>abstract</code>
Derived <code>override</code> never runs	Used <code>new</code> instead of <code>override</code> , or wrong signature	Match the signature exactly and use <code>override</code>
Same field declared in base and derived	Re-declared an inherited field	Delete the duplicate — you already inherited it
<code>List<Track></code> only shows base behavior	Variable typed too specifically, or no <code>virtual / override</code>	The base method must be <code>virtual/abstract</code> and overridden in each kind

Self-Assessment

Skill	Status
I can make one class inherit from another and call <code>base(...)</code>	
I can explain the <code>is-a</code> test for when inheritance fits	
I can use <code>virtual / override</code> correctly	
I can write an abstract class with an abstract method	
I can hold mixed kinds in one base-typed <code>List<></code>	
I can make one loop produce per-type behavior (polymorphism)	
I can explain abstraction and polymorphism and point to my code	

Summary and Tomorrow's Task (17:15 – 17:45)

Today you built a family of related kinds: a shared base that models the idea, derived kinds that fill in the specifics, and one collection that calls one method and gets the right behavior for each object. You named two more pillars — **abstraction** (modeling the idea without the specifics) and **polymorphism** (one call, many behaviors).

Task before tomorrow: An abstract class says *what something is*. But sometimes you want to say only *what something can do* — be sortable, be comparable — without saying what it is, and you may want two unrelated types to share that ability. Write two sentences: how would you describe a pure "capability" that any type could promise to provide, separate from what the type actually is? Tomorrow we meet interfaces, and tie the whole week together.